

Brandon Peck

CSUF Summer AI Chip Design Internship

Assignment 3

Question 1:

Here is the code it generated for the D-Latch and D-FlipFlop DFF Design:

```
// d_flip_flop.v
module d_flip_flop (
    input wire data_in,      // Data input
    input wire clock,        // Clock input
    input wire reset_n,      // Asynchronous active-low reset
    output reg data_out      // Data output
);

    always @(posedge clock or negedge reset_n) begin
        if (!reset_n) begin
            data_out = 1'b0;
        end else begin
            data_out = data_in;
        end
    end
end

endmodule
```

DFF Testbench:

```
// d_flip_flop_tb.v
// Testbench for the d_flip_flop module.

`timescale 1ns / 1ps

module d_flip_flop_tb;

    reg test_data_in;
    reg test_clock;
    reg test_reset_n;
    wire ff_output;

    parameter CLK_PERIOD = 20;

    d_flip_flop dut_ff (
        .data_in(test_data_in),
        .clock(test_clock),
        .reset_n(test_reset_n),
        .data_out(ff_output)
    );

    always begin
        test_clock = 1'b0;
        #(CLK_PERIOD / 2);
        test_clock = 1'b1;
        #(CLK_PERIOD / 2);
    end

    initial begin
```

```

    $dumpfile("d_flip_flop.vcd");
    $dumpvars(0, d_flip_flop_tb);

    test_data_in = 1'b0;
    test_reset_n = 1'b0;

    #10;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #10;
    test_reset_n = 1'b1;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #10;
    test_data_in = 1'b1;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #CLK_PERIOD;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #10;
    test_data_in = 1'b0;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #CLK_PERIOD;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #5;
    test_data_in = 1'b1;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #CLK_PERIOD;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #10;
    test_data_in = 1'b0;
    test_reset_n = 1'b0;
    $display("Time: %0t, data_in=%b, clock=%b, reset_n=%b, data_out=%b",
$time, test_data_in, test_clock, test_reset_n, ff_output);

    #20;
    $display("Simulation finished at time: %0t", $time);
    $finish;
end

```

Endmodule

DL Design:

// d_latch.v

module d_latch (

```

        input wire data_in,      // Data input
        input wire enable_in,   // Enable input
        output reg data_out     // Data output
    );

    always @(data_in or enable_in) begin
        if (enable_in) begin
            data_out = data_in;
        end
    end

endmodule

DL Testbench:
// d_latch_tb.v
// Testbench for the d_latch module.

`timescale 1ns / 1ps

module d_latch_tb;

    reg test_data_in;
    reg test_enable_in;
    wire latch_output;

    d_latch dut_latch (
        .data_in(test_data_in),
        .enable_in(test_enable_in),
        .data_out(latch_output)
    );

    initial begin
        $dumpfile("d_latch.vcd");
        $dumpvars(0, d_latch_tb);

        test_data_in = 1'b0;
        test_enable_in = 1'b0;

        #10;
        test_data_in = 1'b1;
        $display("Time: %0t, data_in=%b, enable_in=%b, data_out=%b", $time,
test_data_in, test_enable_in, latch_output);

        #10;
        test_enable_in = 1'b1;
        $display("Time: %0t, data_in=%b, enable_in=%b, data_out=%b", $time,
test_data_in, test_enable_in, latch_output);

        #10;
        test_data_in = 1'b0;
        $display("Time: %0t, data_in=%b, enable_in=%b, data_out=%b", $time,
test_data_in, test_enable_in, latch_output);

        #10;
        test_data_in = 1'b1;
        $display("Time: %0t, data_in=%b, enable_in=%b, data_out=%b", $time,
test_data_in, test_enable_in, latch_output);
    end
endmodule

```

```

#10;
test_enable_in = 1'b0;
$display("Time: %0t, data_in=%b, enable_in=%b, data_out=%b", $time,
test_data_in, test_enable_in, latch_output);

#10;
test_data_in = 1'b0;
$display("Time: %0t, data_in=%b, enable_in=%b, data_out=%b", $time,
test_data_in, test_enable_in, latch_output);

#10;
test_enable_in = 1'b1;
test_data_in = 1'b1;
$display("Time: %0t, data_in=%b, enable_in=%b, data_out=%b", $time,
test_data_in, test_enable_in, latch_output);

#20;
$display("Simulation finished at time: %0t", $time);
$finish;
end

```

endmodule

Here Are the Timing Diagrams That I requested:

DFF:

```

Time (ns) -> 0   10   20   30   40   50   60   70   80   90  100
-----
CLK (Clock)  : __| ^^^^|____| ^^^^|____| ^^^^|____| ^^^^|____| ^^^^|__
                (Posedge) (Posedge) (Posedge) (Posedge) (Posedge)

D (Data In)  : __|_____|____|_____|____|____|_____|____|____|__
                0       1     0       1     0       1     0       1

Q (Output)   : __|_____|____|_____|____|_____|____|____|____|__
                (Holds)  (Updates) (Holds) (Updates) (Holds)

```

DL:

```

Time (ns) -> 0   10   20   30   40   50   60   70   80   90  100
-----
CLK (Enable) : __| ^^^^^^^^^^^|_____| ^^^^^^^^^^^|_____|
                (Low) (High)   (Low)   (High)

D (Data In)  : __|_____|____|_____|____|____|_____|____|____|__
                0       1     0       1     0       1

Q (Output)   : __|_____|____|_____|____|____|_____|____|____|__
                (Holds) (Transparent) (Holds) (Transparent)

```

Reflection:

If I did not understand the difference between the two already, I don't think this timing diagram would help me very much. It is not very detailed. However, the attached explanation it gave was very good. It did a good job of explaining the concept of the edge triggered D-flipflop and how it is different from the D-Latch. It also included a great graphic showcasing the differences:

Feature	D Latch (Level-Sensitive)	D Flip-Flop (Edge-Triggered)
Triggering	Responds to the <i>level</i> of the enable signal (e.g., when <code>CLK</code> is high).	Responds only to the <i>edge</i> (transition) of the clock signal (e.g., rising edge).
Transparency	Transparent when enabled; <code>Q</code> follows <code>D</code> .	Never transparent ; <code>Q</code> holds its value between clock edges.
Output Update	Output can change multiple times while enable is active.	Output changes at most once per clock cycle (at the active edge).
Complexity	Simpler (fewer gates).	More complex (typically built from two latches in a master-slave configuration).
Use Case	Often used in asynchronous designs or as building blocks for flip-flops.	Essential for synchronous digital systems (registers, counters, state machines).

Question 2:

Here are the generated scripts:

Design:

```
//
-----
// Title: 4-bit Register File (8-bit wide registers)
// Description: Implements a 4-register file with synchronous reset and write enable.
//
-----

module register_file (
    input wire clk,           // Clock signal
    input wire reset,         // Synchronous reset signal
    input wire write_en,      // Write enable signal (active high)
    input wire [1:0] read_addr_a, // Read address for port A (2 bits, selects R0-R3)
    input wire [1:0] read_addr_b, // Read address for port B (2 bits, selects R0-R3)
    input wire [1:0] write_addr, // Write address (2 bits)
    input wire [7:0] write_data, // Data to write (8 bits)
    output wire [7:0] read_data_a, // Data output from port A (8 bits)
    output wire [7:0] read_data_b // Data output from port B (8 bits)
);

    // Define the register memory.
```

```

// The memory is an array of 8-bit registers, 4 elements deep.
reg [7:0] registers [0:3];

//
-----

// Register File Write Logic
//
-----

// This block handles synchronous reset and writing new data into the
register file.
always @(posedge clk) begin
    if (reset) begin
        // Synchronous reset: Clear all registers to 0.
        registers[0] <= 8'h00;
        registers[1] <= 8'h00;
        registers[2] <= 8'h00;
        registers[3] <= 8'h00;
    end else if (write_en) begin
        // Write operation: Only if write_en is high.
        // The write operation is performed at the rising edge of the
clock.
        // We use the write_addr to select the specific register.
        registers[write_addr] <= write_data;
    end
end

//
-----

// Register File Read Logic
//
-----

// Read ports (A and B) are combinational.
// The data is read directly from the register memory based on the read
addresses.

// Assigning data for Port A:
// We use a continuous assignment (assign) to output the data from the
selected register.
// This provides asynchronous reading (data is available immediately based
on the address).
assign read_data_a = registers[read_addr_a];

// Assigning data for Port B:
assign read_data_b = registers[read_addr_b];

endmodule
Testbench:
//
-----
// Title: Testbench for 4-bit Register File
// Description: Self-contained testbench for the register_file module.
//             No external scripts required for simulation.
//
-----

```

```

module tb_register_file;

    //
    -----
    // Testbench Parameters
    //
    -----
    parameter CLK_PERIOD = 10; // Clock period in time units (e.g., 10ns)
    //
    -----
    // Testbench Signals (Inputs to DUT)
    //
    -----
    reg clk;           // Clock signal
    reg reset;         // Synchronous reset signal
    reg write_en;       // Write enable signal
    reg [1:0] read_addr_a; // Read address for port A
    reg [1:0] read_addr_b; // Read address for port B
    reg [1:0] write_addr; // Write address
    reg [7:0] write_data; // Data to write

    //
    -----
    // Testbench Signals (Outputs from DUT)
    //
    -----
    wire [7:0] read_data_a; // Data output from port A
    wire [7:0] read_data_b; // Data output from port B

    //
    -----
    // Instantiate the Device Under Test (DUT)
    //
    -----
    register_file dut (
        .clk(clk),
        .reset(reset),
        .write_en(write_en),
        .read_addr_a(read_addr_a),
        .read_addr_b(read_addr_b),
        .write_addr(write_addr),
        .write_data(write_data),
        .read_data_a(read_data_a),
        .read_data_b(read_data_b)
    );

    //
    -----
    // Clock Generation
    //
    -----
    always #((CLK_PERIOD / 2)) clk = ~clk; // Toggle clock every half period

    //
    -----
    // Initial Block for Test Stimulus

```

```

//
-----
initial begin
    // Initialize all signals
    clk = 0;
    reset = 1;        // Start with reset active
    write_en = 0;
    read_addr_a = 2'b00;
    read_addr_b = 2'b00;
    write_addr = 2'b00;
    write_data = 8'h00;

    // Display header

$display("-----")
;
    $display("Starting Register File Testbench Simulation");

$display("Time\tClk\tReset\tWE\tW_Addr\tW_Data\tR_AddrA\tR_DataA\tR_AddrB\tR_DataB");

$display("-----")
;

//
-----

    // Phase 1: Apply Reset
    //
-----

    // Hold reset for a few clock cycles to ensure proper initialization
    repeat (2) @(posedge clk);
    reset = 0;        // De-assert reset
    $display("%0t\t%b\t%b\t%b\t%b\t%h\t%b\t%h\t%b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

//
-----

    // Phase 2: Write Operations
    //
-----

    // Write 8'hAA to Register 0
    @(posedge clk);
    write_en = 1;
    write_addr = 2'b00; // R0
    write_data = 8'hAA;
    read_addr_a = 2'b00; // Read R0
    read_addr_b = 2'b01; // Read R1 (should be 0 initially)
    $display("%0t\t%b\t%b\t%b\t%b\t%h\t%b\t%h\t%b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

    // Write 8'hBB to Register 1
    @(posedge clk);
    write_en = 1;
    write_addr = 2'b01; // R1

```



```

        write_data = 8'hBB;
        read_addr_a = 2'b00; // Read R0
        read_addr_b = 2'b01; // Read R1
        $display("%0t\t\b\t\b\t\b\t\b\t%h\t\b\t%h\t\b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

        // Write 8'hCC to Register 2
        @(posedge clk);
        write_en = 1;
        write_addr = 2'b10; // R2
        write_data = 8'hCC;
        read_addr_a = 2'b01; // Read R1
        read_addr_b = 2'b10; // Read R2
        $display("%0t\t\b\t\b\t\b\t\b\t%h\t\b\t%h\t\b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

        // Write 8'hDD to Register 3
        @(posedge clk);
        write_en = 1;
        write_addr = 2'b11; // R3
        write_data = 8'hDD;
        read_addr_a = 2'b10; // Read R2
        read_addr_b = 2'b11; // Read R3
        $display("%0t\t\b\t\b\t\b\t\b\t%h\t\b\t%h\t\b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

        //
        -----
        // Phase 3: Read Operations (without writing)
        //
        -----
        @(posedge clk);
        write_en = 0; // Disable write
        write_data = 8'h00; // Data doesn't matter when write_en is 0
        write_addr = 2'b00; // Address doesn't matter when write_en is 0

        read_addr_a = 2'b00; // Read R0 (expect 0xAA)
        read_addr_b = 2'b11; // Read R3 (expect 0xDD)
        $display("%0t\t\b\t\b\t\b\t\b\t%h\t\b\t%h\t\b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

        @(posedge clk);
        read_addr_a = 2'b01; // Read R1 (expect 0xBB)
        read_addr_b = 2'b10; // Read R2 (expect 0xCC)
        $display("%0t\t\b\t\b\t\b\t\b\t%h\t\b\t%h\t\b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

        //
        -----
        // Phase 4: Re-apply Reset and verify
        //
        -----

```

```

        @(posedge clk);
        reset = 1; // Assert reset again
        $display("%0t\t%b\t%b\t%b\t%b\t%h\t%b\t%h\t%b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

        @(posedge clk);
        reset = 0; // De-assert reset
        // All registers should be 0 after reset
        read_addr_a = 2'b00; // Read R0 (expect 0x00)
        read_addr_b = 2'b01; // Read R1 (expect 0x00)
        $display("%0t\t%b\t%b\t%b\t%b\t%h\t%b\t%h\t%b\t%h", $time, clk, reset,
write_en, write_addr, write_data, read_addr_a, read_data_a, read_addr_b,
read_data_b);

        //
-----
        // End Simulation
        //
-----

        @(posedge clk);

$display("-----")
;
        $display("Simulation Finished.");
        $finish; // Terminate simulation
    end

    //
-----
    // Monitoring (Optional, but useful for quick debug)
    //
-----
    // This monitor prints the internal state of the registers whenever they
change.
    // Note: This only works if the simulator supports monitoring internal
signals.
    // It's good for seeing the effect of writes.
    // For formal verification, you'd compare read_data_a/b with expected
values.
    initial begin
        $monitor("Internal Registers: R0=%h R1=%h R2=%h R3=%h",
                dut.registers[0], dut.registers[1], dut.registers[2],
dut.registers[3]);
    end

endmodule

```

After asking some questions about why the AI made its decisions I think that the register file is really well done for the most part. I would clean up the excessive comments, but it has everything necessary. 2 read and write addresses for manipulation; simple reset and write options. Overall it is a really good script.

Question 3:

Here is the corrected code:

```
module DFF (  
    input clk,  
    input d,  
    output reg q  
);  
  
always @(posedge clk) begin  
    q <= d;  
end  
  
endmodule
```

The explanation the AI gave me was that there was a missing semicolon after the port list, and it separated the input declarations for clarity.

Question 4:

Here is the corrected code:

```
module detect_3ones (  
    input clk, reset, x,  
    output reg z  
);  
  
reg [1:0] state;  
  
// Define states for clarity  
parameter S0 = 2'b00; // Initial state, no consecutive ones or sequence broken  
parameter S1 = 2'b01; // First '1' detected  
parameter S2 = 2'b10; // Second '1' detected  
parameter S3 = 2'b11; // Third '1' detected  
  
always @(posedge clk or posedge reset) begin  
    if (reset) begin  
        state <= S0;  
        z <= 1'b0; // Ensure z is reset  
    end else begin  
        z <= 1'b0; // Default to de-assert z  
  
        case (state)  
            S0: begin  
                if (x)  
                    state <= S1;  
                else  
                    state <= S0; // Stay in S0 if input is 0  
            end  
            S1: begin  
                if (x)  
                    state <= S2;  
                else  
                    state <= S0; // Go back to S0 if sequence broken by 0  
            end  
            S2: begin  
                if (x) begin  
                    state <= S3; // Third '1' detected, move to S3  
                    z <= 1'b1; // Assert z  
                end  
            end  
        endcase  
    end  
end
```

```

        end else
            state <= S0; // Go back to S0 if sequence broken by 0
        end
    S3: begin // State to indicate 3+ consecutive '1's
        if (x) begin
            state <= S3; // Stay in S3 if more '1's come
            z <= 1'b1;    // Keep z asserted
        end else begin
            state <= S0; // Go back to S0 if sequence broken by 0
        end
    end
    default: state <= S0; // Should not happen, but good practice
endcase
end
end

endmodule

```

Here is its explanation:

Explanation of Corrections:

- State Definitions:** Added parameters `S0`, `S1`, `S2`, and `S3` for better readability and understanding of the state machine's purpose.
- New State `S3`:** A new state `S3` (representing "three or more consecutive '1's") is introduced.
 - When the FSM is in `S2` and `x` is `1`, it transitions to `S3` and asserts `z`.
 - While in `S3`, if `x` remains `1`, the FSM stays in `S3` and `z` remains asserted. This ensures `z` stays high as long as the `1`s continue.
 - If `x` becomes `0` while in `S3`, the FSM goes back to `S0` (resetting the count of consecutive `1`s).
- Output `z` Control:**
 - `z` is now set to `0` by default at the beginning of the `always` block (`z <= 1'b0;`). This is a common practice for synchronous outputs to ensure it's de-asserted unless explicitly asserted by a state transition.
 - `z` is asserted (`z <= 1'b1;`) only when transitioning to `S3` and while remaining in `S3` (as long as `x` is `1`).
- Reset Logic:** In the reset condition, `z` is also explicitly reset (`z <= 1'b0;`) to ensure a clean start.
- Handling '0' Inputs:** In states `S0`, `S1`, `S2`, and `S3`, if `x` is `0`, the FSM now correctly transitions back to `S0`, indicating that the sequence of consecutive `1`s has been broken.

Question 5:

Here is the generated state diagram

Code snippet

```
graph TD
    S0((S0)) -- 1 --> S1
    S0 -- 0 --> S0
    S1 -- 0 --> S2
    S1 -- 1 --> S1
    S2 -- 1 --> S3
    S2 -- 0 --> S0
    S3 -- 1 --> S4
    S3 -- 0 --> S0
    S4 -- 1 --> S1
    S4 -- 0 --> S2

    S0(S0) -- "Output z = 0" --> S0
    S1(S1) -- "Output z = 0" --> S1
    S2(S2) -- "Output z = 0" --> S2
    S3(S3) -- "Output z = 0" --> S3
    S4(S4) -- "Output z = 1" --> S4
```

Here is the generated state table:

Current State	Input (x) = 0	Input (x) = 1	Next State (x=0)	Next State (x=1)	Output (z)
S0	S0	S1	S0	S1	0
S1	S2	S1	S2	S1	0
S2	S0	S3	S0	S3	0
S3	S0	S4	S0	S4	0
S4	S2	S1	S2	S1	1

 Export to Sheets

Here is the Design:

```
// moore_pattern_detector.v
module moore_pattern_detector (
    input wire clk,
    input wire rst_n, // Asynchronous active-low reset
    input wire x,      // Input bit
    output wire z       // Output (1 when "1011" detected)
);
```

```

// State encoding
parameter S0 = 3'b000; // Initial state, no match
parameter S1 = 3'b001; // Matched "1"
parameter S2 = 3'b010; // Matched "10"
parameter S3 = 3'b011; // Matched "101"
parameter S4 = 3'b100; // Matched "1011"

reg [2:0] current_state;
reg [2:0] next_state;

// State register
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        current_state <= S0;
    end else begin
        current_state <= next_state;
    end
end

// Next state logic
always @(*) begin
    next_state = current_state; // Default to stay in current state
    case (current_state)
        S0: begin
            if (x == 1'b1)
                next_state = S1;
            else
                next_state = S0;
        end
        S1: begin
            if (x == 1'b0)
                next_state = S2;
            else // x == 1'b1
                next_state = S1; // Stays in S1 if another '1' comes (e.g.,
11 -> still matching '1')
        end
        S2: begin
            if (x == 1'b1)
                next_state = S3;
            else // x == 1'b0
                next_state = S0; // Incorrect bit, reset to S0
        end
        S3: begin
            if (x == 1'b1)
                next_state = S4;
            else // x == 1'b0
                next_state = S0; // Incorrect bit, reset to S0
        end
        S4: begin // Pattern "1011" fully matched, now decide next state
based on current input
            if (x == 1'b1)
                next_state = S1; // If '1' after "1011", it starts new "1"
sequence (e.g., 1011**1**)
            else // x == 1'b0
                next_state = S2; // If '0' after "1011", it starts new "10"
sequence (e.g., 101**10**)
        end
    endcase
end

```

```

        end
        default: next_state = S0; // Should not happen
    endcase
end

// Output logic (Moore FSM: output depends only on current state)
assign z = (current_state == S4) ? 1'b1 : 1'b0;

endmodule

Testbench:
// testbench_moore_pattern_detector.v
`timescale 1ns / 1ps

module testbench_moore_pattern_detector;

    // Inputs
    reg clk;
    reg rst_n;
    reg x;

    // Outputs
    wire z;

    // Instantiate the Unit Under Test (UUT)
    moore_pattern_detector uut (
        .clk(clk),
        .rst_n(rst_n),
        .x(x),
        .z(z)
    );

    // Clock generation
    always begin
        #5 clk = ~clk; // 10ns period (100MHz clock)
    end

    initial begin
        // Initialize inputs
        clk = 0;
        rst_n = 0; // Assert reset
        x = 0;

        $display("-----");
        $display("Starting Moore FSM Pattern Detector Testbench");
        $display("Pattern to detect: 1011");
        $display("Time\tclk\ttrst_n\ttx\ttCurrent_State\ttz");
        $display("-----");

        #10 rst_n = 1; // De-assert reset
        @(posedge clk); // Wait for the first positive clock edge after reset
    end

    // Test Case 1: Matching pattern 1011
    $display("\n--- Test Case 1: Matching pattern 1011 ---");
    // Expected sequence: S0 -> S1 -> S2 -> S3 -> S4 (z=1)
    x = 1; #10; // S0 -> S1

```

```

x = 0; #10; // S1 -> S2
x = 1; #10; // S2 -> S3
x = 1; #10; // S3 -> S4 (z should be 1 now)
x = 0; #10; // S4 -> S2 (z should be 0)

// Test Case 2: Overlapping pattern sequences (e.g., 1011011)
$display("\n--- Test Case 2: Overlapping pattern sequences (1011011)
---");
// From S2 (after '1011' then '0')
// Current state is S2 due to previous x=0 from S4
x = 1; #10; // S2 -> S3
x = 1; #10; // S3 -> S4 (z should be 1 again)
x = 0; #10; // S4 -> S2 (z should be 0)
x = 1; #10; // S2 -> S3
x = 1; #10; // S3 -> S4 (z should be 1 again)
x = 0; #10; // S4 -> S2 (z should be 0)

// Test Case 3: Random non-matching sequences
$display("\n--- Test Case 3: Random non-matching sequences ---");
x = 1; #10; // S2 -> S3
x = 0; #10; // S3 -> S0 (reset)
x = 0; #10; // S0 -> S0
x = 1; #10; // S0 -> S1
x = 1; #10; // S1 -> S1
x = 0; #10; // S1 -> S2
x = 0; #10; // S2 -> S0 (reset)
x = 1; #10; // S0 -> S1
x = 0; #10; // S1 -> S2
x = 0; #10; // S2 -> S0
x = 1; #10; // S0 -> S1
x = 0; #10; // S1 -> S2
x = 1; #10; // S2 -> S3
x = 0; #10; // S3 -> S0
x = 1; #10; // S0 -> S1
x = 0; #10; // S1 -> S2
x = 1; #10; // S2 -> S3
x = 1; #10; // S3 -> S4 (z should be 1)

$display("\n--- End of Test ---");
#20 $finish; // End simulation
end

// Monitor for debugging and verification
always @(posedge clk) begin
    $display("%0t\t%b\t%b\t%b\t%s\t\t%b", $time, clk, rst_n, x,
        convert_state_to_string(uut.current_state), z);
end

// Function to convert state value to string for better readability in
output
function string convert_state_to_string(input [2:0] state_val);
    case (state_val)
        uut.S0: convert_state_to_string = "S0";
        uut.S1: convert_state_to_string = "S1";
        uut.S2: convert_state_to_string = "S2";
    endcase
endfunction

```



```
        uut.S3: convert_state_to_string = "S3";
        uut.S4: convert_state_to_string = "S4";
        default: convert_state_to_string = "Unknown";
    endcase
endfunction

endmodule
```

Reflection:

This time I used Google Gemini. It was my first time using a non-openAI model. I used the AI for all designs and testbenches as well as when I encountered any errors. I also did a little testing to see the extent of the knowledge of the AI. For example I asked it a question to test its knowledge of edge triggered conditions and it easily passed the test. I learned that Gemini is extremely thorough and unless told not to, it will include a truly absurd amount of comments in its code, thoroughly explaining every thinking step. You can see this in questions 2 and 5, in question 1 I requested it to simplify the comments to see if it could. I also was able to learn more about register files, for example you need extra read and write addresses so that an ALU can manipulate them. I also learned that the ALU is used in a separate module file and not the same design file as the register. For my results as well as my conversations with the LLM see the links at the end of the document. Overall the AI was extremely helpful and knowledgeable. It was able to explain topics thoroughly. The only challenge I faced was when the AI itself generated a typo in one of its testbench files. It generated 'clk' instead of 'clk'. Nevertheless I was extremely impressed with Gemini. It is absolutely on par if not better in some ways than chatGPT and TitanGPT. One extremely interesting tidbit I noticed is that just like its OpenAI counterparts, Gemini also suggested that I separate the input declarations when I requested it to correct syntax in questions 3 and 4. Overall next time I think I will use Gemini if I need to understand a topic better, and will use ChatGPT when I wish to generate code that I already understand quickly.

Links to Prompts:

Q1: <https://g.co/gemini/share/4cb6f40f6c8e>

Q2: <https://g.co/gemini/share/790147ef002e>

Q3: <https://g.co/gemini/share/5907a2646f0e>

Q4: <https://g.co/gemini/share/c2ec7327c385>

Q5: <https://g.co/gemini/share/835ab056e888>